

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 05 · LECTURE NOTES

Polygon Triangulation

Turning a shape into triangles, which is what everything downstream wants.

MODULE OUTCOMES

- Prove that every simple polygon has a triangulation.
- Implement ear clipping and state its cost.
- Decompose a polygon into monotone pieces by sweep.
- Triangulate a monotone polygon in linear time.
- Handle holes, and explain the art gallery theorem.

1 Existence

EVERY SIMPLE POLYGON HAS A TRIANGULATION

Claim: every simple polygon with $n \geq 4$ vertices has a **diagonal** — a segment joining two non-adjacent vertices and lying entirely in the interior.

Proof. Take the leftmost vertex v (breaking ties by lowest y), with neighbours u and w . If the segment uw lies inside the polygon it is a diagonal and we are done. Otherwise at least one vertex lies inside triangle uvw ; among those, take the vertex x farthest from the line uw . **The segment vx is then a diagonal** — no edge can cross it, because any edge doing so would have an endpoint inside uvw farther from uw than x .

A diagonal splits the polygon into two smaller simple polygons, each with fewer vertices, so the result follows by induction. **The proof is constructive and is therefore also an algorithm**, though a slow one.

Every triangulation of a simple n -gon has exactly $n - 2$ triangles and $n - 3$ diagonals, regardless of which triangulation is found — which follows from the same induction. **This is an excellent cheap correctness check** to assert in any triangulator you write.

2 Ear clipping

An EAR is a vertex v whose neighbours u, w form a diagonal -- equivalently, triangle uvw lies inside and contains no other vertex.

TWO EARS THEOREM: every simple polygon with $n \geq 4$ has at least two non-overlapping ears. This is why the loop cannot stall.

```
while polygon has > 3 vertices:
    find an ear v
    emit triangle (prev(v), v, next(v))
    remove v
    re-test ONLY prev(v) and next(v)
emit the final triangle
```

$O(n^2)$. Optimisation: only REFLEX vertices can block an ear, so test candidates against the reflex set only.

The two ears theorem is the termination argument, and it is worth knowing that it exists. Many implementations are written without it and their authors cannot say why the loop must make progress — which matters, because when the loop *does* stall, the cause is never the polygon.

EAR CLIPPING'S REAL FAILURE MODE IS ROBUSTNESS

The ear test is an orientation predicate plus a sequence of point-in-triangle tests, and point-in-triangle is itself three orientation tests. Everything rests on Module 02.

With inexact predicates the algorithm can find no ear at all — a state the two ears theorem says is impossible, and which therefore has no handling. The polygon is lying to the algorithm about its own geometry.

That is the classic symptom: a triangulator that hangs or produces garbage on one specific mesh out of ten thousand, and works perfectly on everything else. The mesh is almost always one with collinear or near-collinear vertices — which is to say, an ordinary authored mesh (Module 01 §4).

And the usual field fix is to give up after n iterations and emit a triangle fan. That produces overlapping and inverted triangles, which then corrupt the normals, the lighting, the collision mesh, and the physics — moving a loud bug downstream into several quiet ones. Fix the predicate instead; it is less work than debugging the consequences.

3 The $O(n \log n)$ route

A MONOTONE POLYGON TRIANGULATES IN LINEAR TIME

A polygon is **y-monotone** if every horizontal line meets it in a single connected segment — equivalently, walking from the topmost vertex to the bottommost along either boundary chain never moves back up.

Such a polygon triangulates with one stack pass in $O(n)$. Merge the two chains into a single y-ordered sequence, push vertices onto a stack, and whenever the new vertex makes the stack top convex, pop and emit a triangle. Each vertex is pushed once and popped at most once.

And any simple polygon decomposes into y-monotone pieces by a single plane sweep in $O(n \log n)$, using exactly the machinery of Module 04 — a status structure of edges ordered by x , and a priority queue of vertices ordered by y .

So the total is $O(n \log n)$, which is optimal for the decomposition-based approach. The sweep classifies every vertex into one of five types, and **the split and merge vertices are precisely the places where monotonicity fails** and a diagonal must be inserted.

Vertex type	Condition	Sweep action
Start	Both neighbours lie below it, and the interior angle is less than 180° .	Insert its left edge into the status structure and set that edge's <i>helper</i> to this vertex.
Split	Both neighbours below, interior angle greater than 180°. The interior splits here.	Add a diagonal to the helper of the edge immediately to the left , then insert the new edge. This is one of the two places a diagonal is required.
End	Both neighbours above, interior angle less than 180° .	Remove the edge; if its helper was a merge vertex, add that diagonal now.
Merge	Both neighbours above, interior angle greater than 180°. Two parts of the interior join here.	Mark it as the helper of the edge to its left; the diagonal is added later, when that edge is removed or its helper replaced. The mirror image of a split.
Regular	One neighbour above and one below.	Replace the outgoing edge with the incoming one on the left side, or simply update the helper on the right side.

Split and merge vertices are the only ones that break monotonicity, and each receives exactly one diagonal. The symmetry between them is exact: a merge vertex is a split vertex seen upside down, which is why some implementations simply sweep twice in opposite directions and handle only splits.

4 Holes, and how many guards

HOLES: BRIDGE THEM TO THE OUTER BOUNDARY

A polygon with holes is not a simple polygon, so neither ear clipping nor monotone decomposition applies to it directly — both assume a single closed boundary.

The standard technique is to cut a bridge. Take the hole's rightmost vertex, cast a ray to the right, find the first boundary edge it hits, and connect to a visible vertex of that edge. This produces a single boundary that traverses the bridge in both directions.

The bridge is a degenerate zero-width corridor, traversed once in each direction, which merges the hole's boundary into the outer boundary and restores simplicity. Repeat for each hole, processing them in right-to-left order.

The degeneracies here are genuinely nasty. The ray may hit a vertex rather than an edge interior; two bridges may want the same connection; holes may be nested or may touch the boundary; and the resulting polygon has coincident edges that every downstream predicate must tolerate. **Constrained Delaunay triangulation (Module 09) handles holes natively and is usually the better route** when holes are present.

 $\lceil n/3 \rceil$ guards always suffice, and are sometimes necessary

Chvátal's art gallery theorem, for a simple polygon with n vertices. **Fisk's proof is three lines and uses triangulation as a tool**: triangulate the polygon, then 3-colour the resulting triangulation graph — which is always possible, because the dual of a polygon triangulation is a tree, so a greedy colouring never gets stuck. Every triangle then has one vertex of each colour, so placing guards on the least-frequently-used colour class covers every triangle and therefore the whole polygon, using at most $\lceil n/3 \rceil$ guards. **The comb polygon — a row of prongs, each needing its own guard — shows the bound is tight**. It is a good demonstration that triangulation is foundational rather than merely useful: the theorem is about visibility and the proof is about triangles.

WHICH ONE TO USE

Under a few thousand vertices and no holes: ear clipping. It is simple enough to get right, and the quadratic cost is irrelevant at that size. Add the reflex-vertex cache and it is fast enough for most tools.

Large inputs, or performance-sensitive paths: monotone decomposition. $O(n \log n)$, well understood, and the sweep is reusable machinery you will have built in Module 04 anyway.

Holes present, or triangle quality matters: constrained Delaunay (Module 09). It handles holes without bridging, and it produces triangles with good aspect ratios — which matters enormously if the triangles will be used for finite elements, for physics, or for anything interpolated across.

Shipping anything: use a library. earcut (small, fast, used in map rendering), Triangle (quality meshing), or CGAL (everything). **All of them handle degeneracies you have not thought of**, which is the entire argument — and triangulation is a solved problem that is nonetheless easy to get subtly wrong.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
de Berg et al. — Computational Geometry, chapter 3 (polygon triangulation)	The monotone decomposition of §3, with the five vertex types and the helper mechanism done precisely.
Mount — CMSC 754, triangulation and art gallery lectures (free PDF)	Includes Fisk's proof and the comb lower bound.
Mapbox — earcut (free, MIT)	A production ear clipper with hole support , small enough to read in an afternoon. The best source for how the bridging of §4 is actually done.
Shewchuk — Triangle (free)	Quality constrained Delaunay meshing, and the reference for the Module 09 route.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement ear clipping with exact predicates. Assert the $n - 2$ triangle count.
- 2 **Add the reflex-vertex cache** and measure the speedup.
- 3 Build a polygon with many collinear vertices and run a double-based ear clipper on it. Report what happens.
- 4 Implement the five-way vertex classification by sweep.
- 5 Implement monotone decomposition and verify every piece is monotone.
- 6 **Implement linear-time monotone triangulation** with the stack pass.
- 7 Compare ear clipping and the two-phase method as n grows, and find the crossover.
- 8 Implement hole bridging and construct three degenerate cases that break it.
- 9 **3-colour a triangulation** and place art-gallery guards. Verify coverage by sampling.
- 10 Construct the comb polygon and confirm it needs $\lceil n/3 \rceil$ guards.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Prove that every simple polygon has a diagonal.
- 2 How many triangles and diagonals does a triangulation of an n -gon have?
- 3 State the two ears theorem and say what it guarantees.

- 4 What is ear clipping's cost, and what is its real failure mode?
- 5 Define y-monotone and say why it triangulates in linear time.
- 6 Name the five vertex types and say which ones need diagonals.
- 7 How are holes handled, and what goes wrong?
- 8 State the art gallery theorem and sketch Fisk's proof.